

From Rule Learning to Deep Reinforcement Learning: Operational Zone Classification and Human Review Reduction for Wind Turbines

Gard Indrekvam, Maneesh Singh, Anne-Lena Kampen

Department of Computer Science, Electrical Engineering and Mathematical Sciences, Western Norway University of Applied Sciences (HVL), Bergen, Norway

Correspondence: gard.indrekvam@icloud.com

Abstract

Operational zone classification assigns each ten-minute SCADA reading from a wind turbine to one of eight defined operating states, from safety lockdown to full load operation. A companion study produced readable IF-THEN rules using RIPPER and FOIL, reaching 91% and 92% accuracy on an independent 2017 external test set. Two structural limitations remained. The rules require sensor values to be sorted into fixed bins, which causes errors near narrow zone boundaries and each reading is classified without any memory of what was classified ten minutes earlier.

This paper asks whether those limitations can be addressed by supervised classifiers working on raw continuous features and by deep reinforcement learning agents that can carry context from one timestep to the next. Three DRL algorithms (DQN, PPO, A2C) and three supervised classifiers (Random Forest, Gradient Boosting, MLP) are evaluated alongside the rule-based baselines on the same eight-zone problem. RIPPER probability outputs from the companion study are used directly as input features for all models alongside the four raw SCADA signals.

A Random Forest trained on only four raw SCADA signals achieves 100% classification accuracy on both the 2016 internal validation set and the independent 2017 external test set. This shows that the eight zone definitions are completely separable in the raw sensor space and that the 91-92% accuracy of the rule-based models is caused by information lost during the binning step and not by any genuine difficulty in the classification problem. Among the DRL agents, DQN with a physics guardrail reaches 97.7% accuracy on the 2017 external test.

Beyond accuracy, the paper compares two complete deployment pipelines for reducing the number of readings a human operator must review. Pipeline A places a Random Forest meta-model on top of RIPPER and flags 4,189 readings for review (8.0% of all readings). Pipeline B adds DQN as a first stage before the meta-model reducing flagged readings to 1,221 (2.3%), while catching 98.9% of DQN's remaining errors with only 14 false alarms across 52,234 test readings. Adding DQN before the meta-model reduces the human review workload by approximately 70%.

The results in this paper suggests that for deployments where labelled training data is available, a Random Forest or Gradient Boosting classifier would be the most accurate and efficient choice. DRL is the right choice when a solution is needed that can learn without labels or adapt during operation, with DQN being the recommended algorithm of all the ones tested.

Keywords: *wind turbine, operational zone classification, deep reinforcement learning, SCADA, prognostics and health management, meta-model, deployment pipeline*

1. Introduction

1.1 Background

Wind turbines operate in an environment that is constantly changing. Wind speed varies from second to second, the turbine control system adjusts the pitch angle of the blades and generator torque in response, and the machine transitions between physically distinct operating states throughout each day [1]. These states are not just different operating points on a power curve, they reflect fundamentally different control regimes: safety lockdown, transition to or from lockdown, standby, transition to or from standby, free-wheeling, generator engagement, partial load operation, and full load operation. Together these form eight defined operational zones [2]. A ninth state, high-wind safety shutdown above 25 m/s exists, but was not observed in the dataset used in this study.

Knowing which state the turbine is currently in matters for most tasks in prognostics and health management (PHM). Fault detection models, power curve models, performance degradation tracking, and maintenance scheduling all behave differently depending on whether the turbine is in standby, ramping up, or running at full load [3, 4]. Without knowing the current operating state, normal variability caused by the control system can easily be mistaken for abnormal behaviour, which can lead to false alarms or missed fault indications.

A companion study [2] addressed this problem by defining the eight operational zones from physical turbine behaviour and training two rule-based classifiers, Repeated Incremental Pruning to Produce Error Reduction (RIPPER) [5] and First-Order Inductive Learner (FOIL) [6], to identify these zones automatically from Supervisory Control and Data Acquisition (SCADA) sensor data. The rule-based approach produced explicit IF-THEN rules that a domain expert could read, check, and link directly to known turbine physics. RIPPER reached 91% accuracy on an independent 2017 external test set and FOIL reached 92%. Both models held up well when tested on a different turbine one year after training, showing that the zone definitions capture stable physical behaviour rather than patterns specific to one year of data.

1.2 Limitations of the rule-based approach and motivation for this work

Despite the interpretability of the rule-based classifiers, they have two structural limitations that motivate this paper.

First, RIPPER and FOIL require continuous sensor values to be sorted into categories, called bins, before training. This is necessary because both algorithms work on categorical conditions rather than raw numbers. The bin boundaries were set by domain knowledge and stay fixed. Near zone boundaries, sorting into bins introduces errors: a reading that falls just above or just below a bin edge may be sorted into the wrong bin and therefore classified into the wrong zone. The clearest example in this dataset is the boundary between Zone 5 (free-wheeling) and Zone 6 (generator engagement), which is defined by a very narrow rotor speed window of only 0.2 revolutions per minute (RPM), from 11.0 to 11.2 RPM. Readings near this boundary may move back and forth across the bin edge between consecutive 10-minute samples, and RIPPER has no way to resolve this within a single observation.

Secondly, rule-based classifiers treat each observation as completely independent. Each 10-minute SCADA record is classified without any reference to what was classified in the previous record. In reality, a wind turbine does not jump between distant operating states in a single timestep, the transitions are gradual and follow physical constraints. A classifier that knew the turbine was in Zone 6 ten minutes ago would for

example be better to resolve an unclear current reading near the Zone 5/6 boundary. RIPPER has no such memory.

These two limitations together define the questions this paper investigates. Can supervised machine learning classifiers working on continuous, non-binned features recover zone boundaries more accurately than RIPPER? And can a deep reinforcement learning agent, that can carry context from one timestep to the next, use the context to improve classification at the points where RIPPER is struggling?

We focus on RIPPER rather than FOIL as the rule-based baseline throughout this paper. Although FOIL achieves marginally higher overall accuracy on the external test (92% vs. 91%), RIPPER achieves a higher macro-F1 score on the same test (0.91 vs. 0.89), reflecting more balanced performance across all eight zones. RIPPER also outperforms FOIL substantially on the internal validation set (94% vs. 88%) and produces more compact rule sets [2]. For a PHM application where per-zone reliability matters as much as overall accuracy, RIPPER is therefore the stronger and more practical starting point for comparison.

1.3 Supervised machine learning for SCADA classification

Supervised classifiers such as Random Forests have been used for wind turbine tasks including fault detection, power curve modelling, and anomaly detection [3, 7]. These methods work directly on continuous sensor values, do not require manual binning, and can place decision boundaries at the exact threshold that separates two zones rather than at the nearest bin edge.

There is also a specific property of this classification problem worth noting. The zone labels in this study were constructed from physics-based threshold rules, the same rules that define where one zone ends and another begins [2]. This means the correct decision boundary is already fully determined by the data in principle. Whether a machine learning classifier can recover this boundary exactly is therefore also a test of how cleanly and consistently the zone definitions were specified.

1.4 Deep reinforcement learning for sequential industrial monitoring

Deep reinforcement learning (DRL) is a different approach to classification. Instead of learning from a fixed labelled dataset all at once, a DRL agent interacts with an environment step by step. At each step the agent receives a reward signal based on whether its output was correct or not, and it updates its decision policy over time. The key difference from standard supervised learning is that the agent can observe its own previous output as part of its current input, giving it explicit memory of what it predicted at the most previous timestep.

DRL has been applied to wind turbine control tasks before, most notably for optimising generator torque and blade pitch angles to improve power output [8]. Its use for operational zone classification is less explored. The main argument for applying it here is not that it will be more accurate than a well-trained supervised classifier, but that it can directly use temporal context, specifically the previous zone prediction, when making its current decision. This is exactly the kind of information that could resolve unclear readings near the Zone 5/6 boundary.

Three DRL algorithms are evaluated in this paper: Deep Q-Network (DQN) [9], Proximal Policy Optimisation (PPO) [10], and Advantage Actor-Critic (A2C) [11]. All three are implemented using the Stable-Baselines3 library [12] and trained in a custom environment built on Gymnasium [13]. Each algorithm takes a different approach to learning: DQN is value-based and learns by storing and replaying

past experiences, PPO directly optimises the action selection policy with a stability constraint to keep updates from being too large, and A2C combines a policy and a value estimate in a single update step.

1.5 From accuracy to operator workload: the pipeline comparison

Beyond asking which model is most accurate, this paper asks a question that is more directly relevant to real deployment: how many readings does a human operator need to have a look at?

Even a model with 97% accuracy produces over 1,400 errors on a dataset of 52,000 readings. The question is whether those errors can be identified automatically, so that only the readings that are likely wrong are sent for a human review and the rest are accepted automatically. A secondary model trained to flag likely errors on top of the main classifier is called a meta-model in this paper.

Two complete deployment pipelines are built and compared on the full 2017 external test set of 52,234 readings.

Pipeline A is the direct extension of the rule-based approach from [2]. RIPPER makes predictions, and a Random Forest meta-model trained on RIPPER's own probability outputs flags the readings where RIPPER is likely to be wrong. At a decision threshold of 0.50, Pipeline A flags 4,189 readings for human review, which is 8.0% of all readings.

Pipeline B is the full system introduced in this paper. DQN corrects most of RIPPER's errors first. A LightGBM [14] meta-model then flags only the readings where DQN is still likely to be wrong. At the same 0.50 threshold, Pipeline B flags 1,221 readings, which is 2.3% of all readings, while catching 98.9% of DQN's remaining errors with only 14 false alarms across the entire 52,234-reading test set.

Adding DQN before the meta-model therefore reduces the human review workload by approximately 70% compared to building the meta-model directly on top of RIPPER. This reduction in operator workload and not the gain in raw accuracy, is the central practical finding of this paper.

1.6 Contributions

The specific contributions of this paper are:

- (1) A comparison of three DRL agents (DQN, PPO, A2C) and three supervised classifiers (Random Forest, Gradient Boosting, Multi-Layer Perceptron (MLP)) against the rule-based baselines from [2] on the same eight-zone classification problem, evaluated on both an internal validation set and an external test set from a different turbine at the same site.
- (2) The finding that a Random Forest trained on only four raw SCADA signals achieves 100% classification accuracy on both datasets. This confirms that the eight zone definitions from [2] are cleanly separable in the raw sensor space, and shows that the 91-92% accuracy of RIPPER and FOIL is caused by information lost during binning, not by any underlying overlap between zones.
- (3) An evaluation of RIPPER zone probabilities as input features for all downstream models, showing how RIPPER's internal confidence signal can be reused as an interpretable input to both DRL agents and supervised classifiers.
- (4) A demonstration that DQN with physics guardrails reaches 97.7% accuracy on the external test, needing almost no corrections by the guardrail, showing that the agent has learned the physical zone boundaries largely on its own.

- (5) A comparison of two complete deployment pipelines showing that adding DQN before the meta-model reduces readings flagged for human review from 4,189 (8.0%) in Pipeline A to 1,221 (2.3%) in Pipeline B, a reduction of approximately 70%, while maintaining 98.9% recall of DQN's remaining errors and producing only 14 false alarms across all total test readings.

2. Methods

2.1 Data

The dataset used in this study is the same as in [2]. It consists of ten-minute averaged SCADA measurements from Wind Turbine 7 of an offshore wind farm, provided by EDP (Energias de Portugal). The 2016 data is split chronologically: the first 80% forms the training set and the last 20% forms the internal validation set. The full 2017 dataset from Wind Turbine 11 is used as an external test set to measure how well models trained on 2016 data perform one year later without retraining and parameter adjustments.

Zone labels were produced by the physics-informed labelling procedure from [2], which assigns each observation to one of eight operational zones based on threshold rules applied to wind speed, rotor RPM, blade pitch angle, and power output. The training set contains 41,860 samples, the internal validation set contains 10,466 samples, and the 2017 external test set contains 52,234 samples.

2.2 Feature representation

All models in this study use some combination of the following input features. The four raw SCADA signals are wind speed (Amb_Wind_Speed), rotor RPM (Rtr_RPM_Avg), blade pitch angle (Blades_Pitch_Angle), and grid power output (Grid_Prod_Power). They are all normalised by some fixed upper limits (25 m/s, 15 RPM, 90 degrees, 2000 kW) so that all values fall roughly in the range of 0 to 1.

In addition, the RIPPER model from [2] was used to generate zone probability estimates. Eight separate one-vs-rest RIPPER classifiers were trained, one per zone. Each classifier outputs the probability that the current observation belongs to its assigned zone. This gives eight additional features (ripper_proba_Zone_1 to ripper_proba_Zone_8), plus a margin feature defined as the difference between the highest and the next highest probability. A low margin means RIPPER is uncertain between two zones, which tends to happen near zone boundaries. The full feature set is therefore 13 features: four raw SCADA signals and nine RIPPER-derived features. Because the classifiers are retrained for probability output, the RIPPER and FOIL accuracy figures reported in this paper reflects the re-implementation and slightly differ from the numbers in [2].

To measure how much the RIPPER meta-features actually contribute, the supervised classifiers were also tested using only the four raw SCADA signals. Which allowed for a direct comparison of whether the added RIPPER information changed the accuracy.

2.3 Supervised baselines

Three supervised classifiers were trained as baselines. The Random Forest (RF) [15] used 300 decision trees with balanced class weights and no maximum depth limit. Balanced class weights mean that the training loss gives more weight to observations from smaller zones, so rare zones like Zone 2, with only 97 samples, is not ignored during the training.

The Histogram Gradient Boosting classifier (GBT), a LightGBM-inspired [14] histogram-based implementation of gradient boosting [16] provided by scikit-learn used 300 iterations, a learning rate of 0.05 and a maximum tree depth of 8. Gradient boosting builds trees one at a time, where each new tree is trained to correct the errors left by the previous ones.

The MLP used two hidden layers of 256 units each with ReLU activations, trained with the Adam optimiser [17]. Early stopping was applied based on a 10% internal validation split with a maximum of 500 training epochs. All three classifiers were trained on the full 13-feature set. RF and MLP were also trained on the 4 feature raw SCADA set to isolate the effect of the RIPPER meta-features. All supervised classifiers were implemented using scikit-learn [18].

2.4 DRL environment

The DRL agents were trained in a custom Gymnasium [13] environment called RegimeEnv. Each training episode samples 512 consecutive timesteps from the 2016 training dataset at a random position in the time series. At each step the agent observes the current turbine state and must select one of eight actions, each corresponding to one of the eight zones.

The observation vector has 14 values. The first 13 are the features described in Section 2.2. The 14th is the agent's own previous action. This previous action feature allows the agent to take into account what zone it predicted in the previous step, which gives it a simple form of memory.

The reward function encourages both accuracy and smooth zone transitions:

- A correct prediction receives a reward of +2.
- An incorrect prediction receives a base penalty of -1, plus an additional -0.15 for each zone step away from the correct answer. For example: predicting Zone 3 when the zone actually is Zone 6 gives:
 $-1 - (0.15 \times 3) = -1.45$.
- Any zone jump of two or more zones compared to the previous timestep receives an additional penalty of -0.2 multiplied by the size of the jump. A jump of 2 costs -0.4. This discourages large zone transitions that would not be physically possible in a single 10-minute interval.

The smoothness penalty reflects a real property of wind turbine operation: a turbine in Zone 5 at one timestep is very unlikely to be in Zone 8 at the next one. Having this in the reward functionality pushes the agent towards predictions that match how a real wind turbine moves between the zones. The specific reward values were determined through pilot experiments on the training dataset. They were not tuned systematically and other values might work equally.

2.5 Physics guardrails

Two hard rules derived from turbine physics were applied to the output of each DRL agent after prediction. If the blade pitch angle is 86 degrees or above, the turbine is in safety lockdown and the agent's prediction is overridden to Zone 1 since it is impossible that it could be in any other one. If the blade pitch angle is between 24 and 86 degrees, the turbine is in a transition state and the output is overridden, Zone 2.

Physics guardrails are not applied to the supervised classifiers, because those models already learn these boundaries directly from the training data without any need for post-prediction corrections.

2.6 DRL agents

Three DRL algorithms were trained using Stable-Baselines3 [12]. Each was trained for 200,000 environment steps on the 2016 training data. A callback evaluated each model every 20,000 steps on the 2016 internal validation set and saved a checkpoint for the best performing.

DQN (Deep Q-Network) [9] learns a function that maps states and actions to expected total reward. It uses an experience replay buffer, which stores past transitions and samples from them randomly during training. A separate target network is updated every fixed number of steps to stabilise learning. DQN used a mini-batch size of 128, a learning rate of 0.0001 and a replay buffer of 100,000 transitions.

PPO (Proximal Policy Optimisation) [10] directly optimises the agent's action selection policy, the learned rule maps what the agent currently observes to which zone it predicts. It uses a built-in limit on how much the policy is allowed to change in a single update step, which makes training more stable. It used a learning rate of 0.0003, 2,048 steps per update, mini-batches of 64 and 10 update rounds for each batch.

A2C (Advantage Actor-Critic) [11] updates both a policy and a value function at the same time using the advantage, the difference between the reward actually received and what the value function expected. A2C used a learning rate of 0.0003 and 32 steps per update. All three agents used a network architecture of two hidden layers with 256 units each.

2.7 Meta-models and pipeline comparison

Two meta-models were trained, one for each deployment pipeline. In Pipeline A, a Random Forest meta-model was trained to predict whether RIPPER would be correct on a given observation. It was trained on the 2016 internal validation set using the 13 input features. The target label for each observation is simply whether RIPPER's prediction matched the correct zone. At deployment time, observations where the meta-model predicts low confidence in RIPPER are flagged for review.

For Pipeline B, a LightGBM meta-model was trained to predict whether DQN would make an error. It was also trained on the 2016 internal validation set and tested on the 2017 external set. In addition to the same 13 features used in Pipeline A, this meta-model includes temporal features coming from the DQN prediction sequence: the three most recent DQN zone predictions, the one-step change in each SCADA signal and a rolling measure of how much DQN's predictions have varied over the last five timesteps. These temporal features capture situations where DQN is switching zones rapidly or where the sensor signals are changing quickly, both of which are associated with higher error rates. Both of the meta-models use a decision threshold of 0.50, the standard default for binary classifiers. Other values between 0.3 and 0.9 were tested and produced no meaningful differences in the result.

Figure 1 provides an overview of both pipelines.

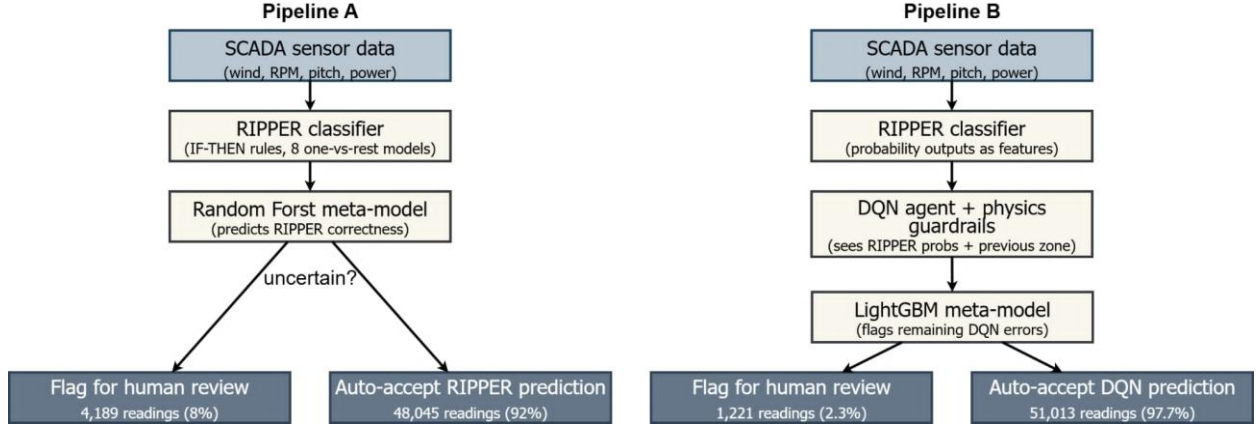


Figure 1. Overview of the two deployment pipelines.

2.8 Evaluation

All models were evaluated on the 2016 internal validation set and the 2017 external test set. Models are assessed using overall accuracy and macro-averaged F1 score. Macro-averaged F1 gives equal weight to all eight zones regardless of how many observations each contains, which makes it sensitive to errors on small zones like Zone 2.

For the DRL agents, two additional metrics were computed: average zone jump (the average absolute difference in zone index between consecutive predictions) and large jumps (the count of timesteps where the predicted zone changed by two or more steps in a single interval). Three metrics were reported on the 2017 external test set for the pipeline comparison: readings flagged for human review, recall of actual errors among flagged readings and the amount of false alarms.

3. Results

3.1 Overall classification performance

Table 1 shows the accuracy and macro-F1 score for all models on both evaluation sets. Random Forest on four raw SCADA signals achieves 100% accuracy on both the validation set and the external test. The Random Forest on the full 13-feature set makes a single error on the validation set (accuracy 99.99%) and achieves 100% on the external test. The Gradient Boosting classifier sits just below at 99.9% on both. Among the DRL agents, DQN with guardrails reaches 97.3% on the validation set and 97.7% on the external test. PPO with guardrails reaches 96.0% on both. A2C with guardrails reaches 92.7% on validation and 93.4% on the external test.

RIPPER reaches 92.6% on the validation set and 91.2% on the external test, and FOIL reaches 92.3% and 91.4% respectively, broadly consistent with the results reported in the companion study [2]. See Figure 2 for a visual comparison of accuracy across all model families.

Table 1. Classification results for all models.

Model	Features	Validation 2016 Acc / F1	External 2017 Acc / F1
RF	Full (13)	99.99% / 0.999	100.0% / 1.000
GBT	Full (13)	99.9% / 0.999	99.9% / 0.999
MLP	Full (13)	99.4% / 0.992	99.4% / 0.993
RF	Raw (4)	100.0% / 1.000	100.0% / 1.000
MLP	Raw (4)	99.5% / 0.980	99.6% / 0.982
DQN + guardrails	Full (13)	97.3% / 0.967	97.7% / 0.968
PPO + guardrails	Full (13)	96.0% / 0.954	96.0% / 0.953
A2C + guardrails	Full (13)	92.7% / 0.847	93.4% / 0.847
DQN (no guardrails)	Full (13)	97.3% / 0.963	97.6% / 0.964
PPO (no guardrails)	Full (13)	95.5% / 0.910	95.6% / 0.909
A2C (no guardrails)	Full (13)	90.1% / 0.578	91.5% / 0.584
RIPPER	Rule-based	92.6% / 0.916	91.2% / 0.914
FOIL	Rule-based	92.3% / 0.897	91.4% / 0.896

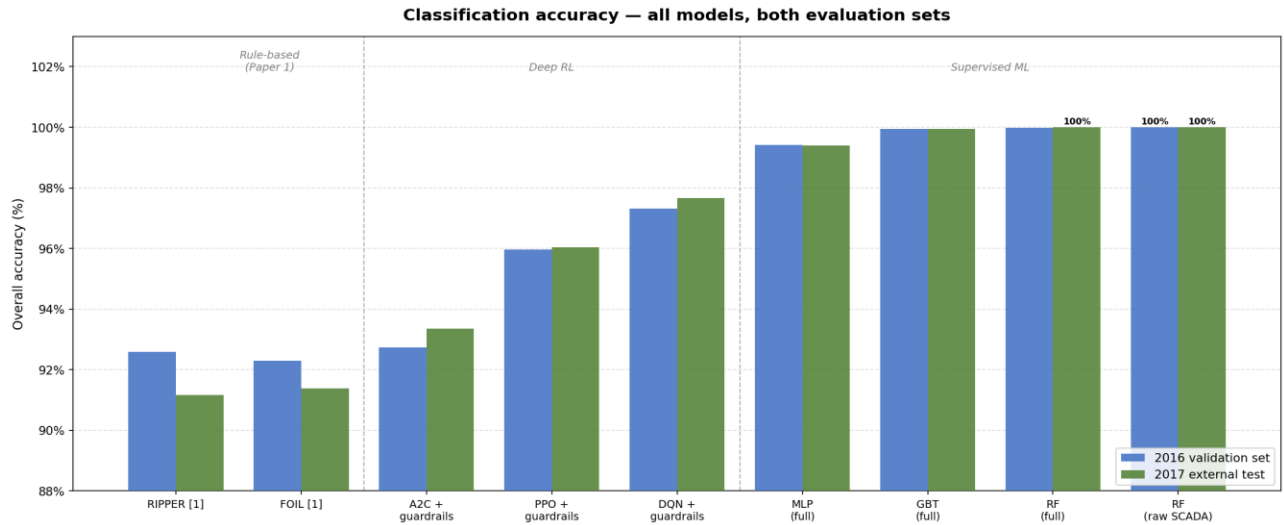


Figure 2. Accuracy comparison across all model groups on the validation set and external test set.

See Figure 3 for the confusion matrices of RIPPER and DQN on the 2017 external test set side by side.

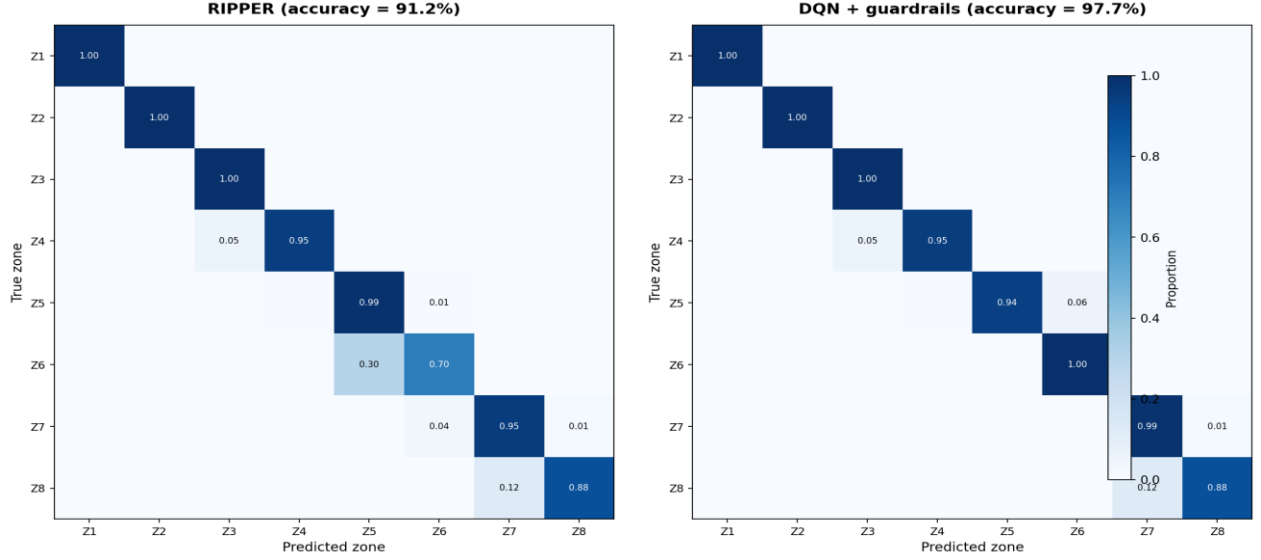


Figure 3. Confusion matrices for RIPPER and DQN with guardrails on the 2017 external test set.

3.2 The 100% Random Forest result

The finding that a Random Forest achieves 100% accuracy using only four raw SCADA signals is an unexpected result in this paper. It means that the eight operational zones defined in [2] are completely separable using the main sensor readings, with no overlap or uncertainty anywhere in the dataset. The RF on raw signals draws the correct zone boundaries without a single error. The RF on the full 13-feature set makes only one error on the validation set and is also perfect on the external test.

This result also puts the RIPPER and FOIL accuracy from [2] into context. Those models reach only 91-92% not because the zones are difficult to separate, but because rule-based algorithms are constrained to produce human-readable IF-THEN rules using binned feature values, introducing errors near zone boundaries as described in Section 1.2. A Random Forest uses the raw continuous values and can find the exact threshold at which one zone ends and another begins. RIPPER and FOIL are limited by the resolution of the bins. The interpretability of the rule-based models is still valuable, and the accuracy gap is not a property of the data, but the algorithm itself and the binning step.

The result also shows that the RIPPER meta-features do not add accuracy for the supervised classifiers. The RF achieves 100% with four raw features and near perfect accuracy with thirteen features, there was only one error on the validation set, meaning the extra RIPPER probabilities contribute almost nothing to the final accuracy. The raw SCADA signals already contain all the information needed to separate the eight zones.

3.3 Effect of physics guardrails

Table 2 shows how often each DRL agent’s prediction on the internal validation set was overridden by the guardrails, and how this changed the macro-F1 score.

Table 2. Physics guardrail correction rates on the validation set.

Agent	Corrections	Correction rate	Macro-F1 gain
DQN	6 / 10,466	0.06%	0.004
PPO	53 / 10,466	0.51%	0.044
A2C	273 / 10,466	2.61%	0.269

For DQN, the guardrails corrected only 6 predictions out of 10,466. This means DQN already respects the physical boundaries almost perfectly on its own. The guardrail correction barely affects its accuracy.

For PPO, the correction rate was higher at 0.51%, and the macro-F1 improved from 0.910 without guardrails to 0.954 with them. A2C showed the largest effect where 2.61% of predictions were overridden on the validation set and the macro-F1 improved from 0.578 to 0.847.

The A2C result is the most informative. Without guardrails A2C produces near-zero recall for Zone 8 that makes up about 5.5% of the validation data. The guardrails cover only Zones 1 and 2, so A2C’s failure on Zone 8 is not automatically corrected. Even with guardrails, A2C’s macro-F1 of 0.847 remains well below DQN and PPO, making A2C a poor choice for this task.

The accuracy gap between RIPPER and DQN is most visible at the Zone 5/6 boundary. Figure 4 shows where errors fall for each model: grey points are correctly classified Zone 5 and Zone 6 readings, red points are misclassified. RIPPER’s 2,768 errors spread across the full Zone 6 band from RPM 11.0 to 11.2, because its fixed bin boundaries cannot resolve that 0.2 RPM window. DQN reduces this to 288 errors, concentrated at the exact 11.0 threshold, the readings that even temporal context cannot resolve.

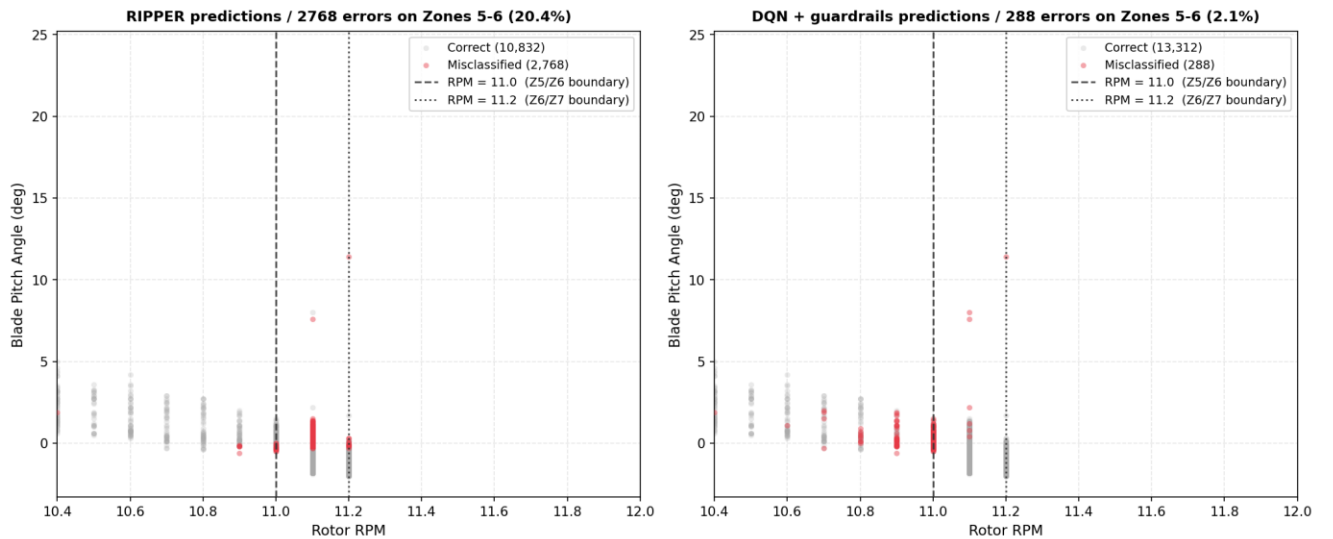


Figure 4. Zone 5/6 boundary errors on the 2017 external test set (grey = correct, red = misclassified).

3.4 Temporal consistency

Table 3 shows how smoothly each model moves through zones over time, measured on the internal validation set from 2016.

Table 3. Temporal consistency metrics on the validation set.

Model	Avg zone jump	Max jump	Large jumps (2+)
DQN + guardrails	0.24	6	233
PPO + guardrails	0.25	6	253
A2C + guardrails	0.21	6	242
A2C (no guardrails)	0.18	4	167
MLP (full features)	0.23	7	223
RIPPER [2]	0.25	6	246

All models show similar average zone jump values between 0.18 and 0.25. The smoothness penalty in the DRL reward function was intended to make the DRL agents smoother than the MLP, which predicts each timestep independently. In practice, the MLP is comparable to DQN and PPO, so the reward penalty did not produce a clear advantage in measured smoothness.

A2C without guardrails shows the lowest average jump of 0.18, but also the lowest accuracy. This combination suggests that A2C avoids large jumps not by learning correct zone transitions but simply by staying in a small number of common zones rather than exploring the full range of the prediction space.

3.5 Cross-year generalisation

Most models maintain or slightly improve their accuracy when moving from the validation set to the external test. The DRL agents show the clearest improvement: DQN moves from 97.3% to 97.7%, and A2C from 92.7% to 93.4%. The supervised classifiers remain stable within 0.1 percentage points. RIPPER drops from 92.6% to 91.2% and FOIL from 92.3% to 91.4%, a drop of around 1-1.5 percentage points for both, which is a small shift and does not affect the practical usefulness of the rule-based models.

The stability across years reflects the nature of the zone definitions. Because the zones are defined by physical thresholds rather than statistical patterns, any model that correctly learns these thresholds should work equally well in any year from the same turbine site as long as the underlying physics has not changed much. The results show that even between two different turbines the zone boundaries are stable enough that no retraining is needed.

3.6 Pipeline comparison

Table 4 shows the full results of the two deployment pipelines on the external test set.

Table 4. Pipeline comparison on the external test set.

Metric	Pipeline A (RIPPER + RF)	Pipeline B (DQN + LightGBM)
Model accuracy	91.2%	97.7%
Errors before flagging	4,614	1,221
Readings flagged	4,189	1,221
Percentage of data flagged	8.0%	2.3%
Errors caught (recall)	4,189 (90.8%)	1,207 (98.9%)
False alarms	0	14

Pipeline A flags 4,189 readings (8.0% of all data), catches 90.8% of RIPPER’s 4,614 errors, and produces 0 false alarms.

Pipeline B flags 1,221 readings (2.3% of all data), catches 98.9% of DQN’s 1,221 errors, and produces 14 false alarms.

The reduction from 4,189 to 1,221 flagged readings represents a 70.9% drop in human review workload. Pipeline B also has higher recall, 8.1% difference, at the cost of introducing 14 false alarms that Pipeline A does not have.

The reason Pipeline B is more efficient is that DQN eliminates most of the easy errors first. DQN has already classified the data and the remaining errors are a much smaller and more concentrated set, mostly located near the narrow zone boundaries where even the best model struggles. The LightGBM meta-model can then target this smaller set more precisely, flagging 98.9% of the remaining errors with only 14 false alarms across 52,234 test readings. Figure 5 summarises these three metrics side by side.

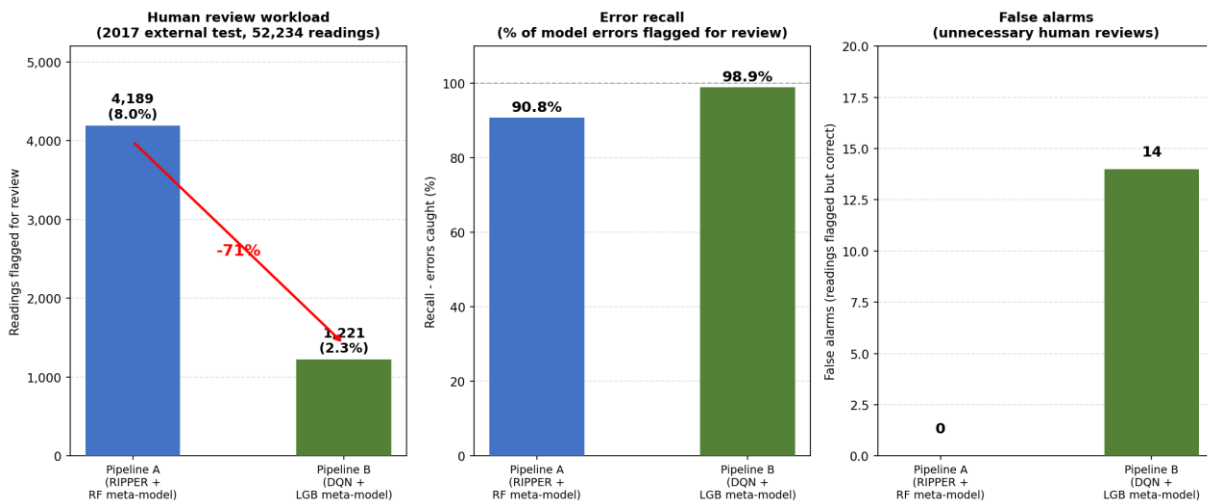


Figure 5. Comparison of Pipeline A and Pipeline B on three metrics: readings flagged for human review, recall of actual errors, and number of false alarms.

4. Discussion

4.1 What the 100% Random Forest result means

The 100% accuracy of the Random Forest on four raw SCADA signals is a strong positive result for the zone framework introduced in [2]. It shows that the eight zones are not just physically motivated, they are also perfectly separated in the sensor space. A Random Forest with 300 trees can draw the correct zone boundaries without a single error across all 10-minute data points spanning two years and two turbines.

For anyone deploying this in practice, zone identification for this turbine type can be done with a very simple and fast classifier. A Random Forest predicts the zone for a new observation extremely fast and only needs the four standard SCADA signals as input.

It also confirms something important about the zone framework from [2]: the physics-informed threshold rules that define the zones did not introduce any overlap between zones in the raw sensor space. Every observation in the dataset sits cleanly on one side of a zone boundary, which means the labelling approach from [2] worked as intended.

The implication for RIPPER and FOIL is equally clear. The accuracy gap between their 91-92% and the Random Forest's 100% is entirely explained by the binning step. When RIPPER or FOIL bins a reading near a zone boundary, that reading may end up in the wrong bin and no amount of rule learning can correct it, which the Random Forest avoids totally by working with only the raw numbers.

This does not mean RIPPER is the wrong tool. For a PHM deployment where the operator needs to be able to read and explain every classification decision, an IF-THEN rule is far more useful than a forest of 300 decision trees. The 8-9% accuracy cost is the price of full interpretability. Whether that trade-off is worth it is a decision for the operator, not the algorithm.

4.2 When DRL is still the right choice

If a supervised classifier with 100% accuracy already exists, why train DRL agents at all? The answer is about the conditions under which a model is deployed, not about its accuracy on a known dataset.

A supervised classifier requires a complete labelled training dataset before training can begin. Labelling requires a domain expert who understands the zone definitions well enough to verify that each data point has been correctly assigned. For a new turbine type, farm, or a turbine that has changed behaviour after a big repair, the labelling process must start from scratch.

A DRL agent does not need labelled data in the same way as a supervised classifier, though in this study the reward was still computed from zone labels. The theoretical advantage is demonstrated by Zone 1 and Zone 2, where physics guardrail rules provide automated corrections without any manual labelling. If the reward function could be extended to cover all eight zones using physics rules alone a DRL agent could be deployed on a new turbine and learn the correct zone boundaries without any human labelling effort, which is a practical advantage for scaling zone classification to large wind farms with different turbine types.

A second argument for DRL is online adaptation. Wind turbine behaviour changes as the turbine ages: bearings wear, pitch control tolerances shift, and power output gradually declines [3, 4]. A static supervised classifier trained in 2016 does not automatically adjust to these changes in 2026. A DRL agent that

continues to receive rewards during deployment can update its policy to track these shifts without needing a new labelled dataset. The 2016 to 2017 generalisation results in this study show that the changes between years are small enough that even a static model works well over a one-year horizon, but longer time horizons or more significant hardware changes could reveal this limitation.

For DRL applied to wind turbine tasks, existing work has largely focused on control problems such as pitch angle optimisation and torque control to improve power output [8]. The results in this study suggest that the same DRL tools can be applied to the classification problem with good success, opening up the possibility of a single learning framework that handles both classification and control adaptation.

The recommendation from this study is: use a supervised classifier, preferably a Random Forest, when labelled training data is available and the turbine behaviour is well understood. Use DRL when a solution is needed that can learn without labels or adapt after deployment, and accept that it will cost some percentage points in accuracy compared to the best supervised classifier. DQN with guardrails is the recommended choice among the three DRL agents tested.

4.3 Comparison of the three DRL agents

DQN performs clearly best of the three algorithms tested. Its 97.7% accuracy on the 2017 external test is the closest to the supervised baselines, its guardrail correction rate is very low (0.06%), and its per-zone performance is consistent across all eight zones. A possible explanation to why DQN outperforms both PPO and A2C is that its experience replay buffer allows it to learn from rare zone transitions that would otherwise be seen too little during the training.

PPO is a reasonable second choice, particularly after guardrail correction. Its accuracy of 96.0% on the external test is 1.7 percentage points below DQN, which is a meaningful but not dramatic difference for most PHM applications.

A2C is not recommended for this task. Without guardrails it fails Zones 1, 2 and 8 entirely. The guardrails correct Zone 1 and Zone 2, while the Zone 8 failure persists, making A2C unsuitable for any application that requires reliable classification across all the different operational regimes. Zone 8 represents above-rated wind speed operation [2], where the turbine is producing at maximum power and where control actions are most critical for safety. A model that consistently misclassifies Zone 8 would give incorrect context to every fault detection or power curve model that relies on knowing the current zone. The guardrails improve A2C significantly on Zones 1 and 2, but they cannot fix the Zone 8 failure, which leaves A2C's macro-F1 at 0.847, quite a lot below the other agents.

4.4 The pipeline comparison in context

The pipeline comparison is the most practically relevant result in this paper for anyone considering deploying zone classification in a real wind farm.

The key insight is that the DRL agent and the meta-model serve different functions. The DRL agent (DQN) fixes most of the errors that RIPPER makes, reducing the total error count from 4,614 to 1,221 on the 2017 external test. The LightGBM meta-model then catches 98.9% of the remaining 1,221 errors, leaving only 14 false alarms and 14 missed errors out of 52,234 readings.

The result is that Pipeline B requires a human operator to review only 1,221 readings, compared to 4,189 for Pipeline A. This is a reduction of 70.9% in review workload, which directly translates to reduced time spent on manual verification in a live monitoring system.

The 14 false alarms in Pipeline B are also worth noting. This means that 1,207 of the 1,221 flagged readings are genuine errors that need correction, and only 14 are readings where the model was actually right and flagged unnecessarily. Pipeline A produces 0 false alarms, but at the cost of lower recall and more than three times more readings flagged for review.

One limitation of this comparison is that it assumes a human operator who reviews a flagged reading will always correct it perfectly. In real world practice some flagged readings near zone boundaries may be genuinely difficult to assign even for a human expert.

5. Limitations

All results in this paper come from a single offshore wind farm, evaluated on data from two turbines over two years. Whether the findings hold for other turbine types, onshore installations, or turbines with fundamentally different control systems is an open question. The 100% Random Forest accuracy result in particular may not replicate if the zone definitions need to be adapted for a different turbine model since the zone boundaries could sit in different positions in the sensor space.

The DRL agents were trained in an offline batch setting. They received rewards computed from historical labelled data, not from a live running turbine. True live learning where the agent interacts with a real turbine and updates its policy during operation was not tested. The practical benefits of DRL for continuous adaptation therefore remain theoretical in the context of this study. In this study the reward was still computed from zone labels during training, so the label-free advantage remains a direction for future work rather than a result demonstrated here.

The study does not include a formal statistical test for the differences in accuracy shown in Table 1. With 52,234 external test readings, a 1% accuracy difference corresponds to roughly 500 observations, so the larger gaps are almost certainly meaningful in practice, but no formal significance test was applied.

The DRL agents were not tested without the RIPPER meta-features as inputs. It is unknown whether removing these features would give a significant change in performance. Testing this is a natural next step. Observation feature number 14, the previous action of the agent, gives DRL agents a form of temporal memory not shared by any supervised classifier in this study. Whether an MLP augmented with lagged predictions as extra inputs would close the gap with DQN was not tested either.

All models were evaluated on zone classification accuracy only. A complete PHM evaluation would also measure whether correct zone identification actually improves downstream tasks such as fault detection rate, false alarm rate in fault detection, or power curve modelling error. This kind of end-to-end evaluation is an important direction for future work.

The pipeline comparison assumes a fixed decision threshold of 0.50 for both meta-models. Adjusting this threshold moves the trade-off between flagged volume and recall. A deployment with very low tolerance for missed errors could lower the threshold to flag more readings, at the cost of more false alarms.

Lastly, the meta-models were trained on the 2016 internal validation set and tested on the 2017 external set from a different turbine at the same wind farm. Since both turbines operate at the same site under the

same environmental conditions, this cross-turbine test does not fully capture how the meta-models would perform on a turbine from a different farm or climate region. Whether the meta-models generalise to turbines at completely different sites with other operating histories has not been assessed.

6. Conclusion

This paper compared deep reinforcement learning agents and supervised classifiers for wind turbine operational zone classification, using the eight-zone framework from the companion study [2].

The main findings are as follows.

A Random Forest trained on four raw SCADA signals achieves 100% accuracy on both the 2016 internal validation set and the 2017 external test set. This confirms that the zone definitions from [2] are completely separable in the raw sensor space and validates the quality of the physics-informed labelling approach. It also explains why RIPPER and FOIL from [2] reached only 91-92% since the accuracy gap is caused by information lost during the binning step which is required by rule-based algorithms, and not by any genuine difficulty in the classification problem.

Among the DRL agents, DQN with physics guardrails reaches 97.7% accuracy on the 2017 external test. PPO with guardrails reaches 96.0%. A2C is significantly weaker at 93.4% and shows a specific failure on Zone 8 without guardrails, which makes it unsuitable for PHM applications where that zone carries the most safety-critical context.

Physics guardrails are very effective for A2C and PPO, correcting up to 2.61% of predictions and substantially improving the macro-F1 score. For DQN, they make almost no difference because the agent already learns the physical zone boundaries on its own.

RIPPER zone probabilities as input features do not improve supervised classifier accuracy, since the raw SCADA signals alone are sufficient. Their value for DRL state representation is that they give the agent interpretable confidence signals about what zone RIPPER thinks the turbine is currently in.

All models generalise well from 2016 to 2017 without retraining, supporting the view that the zone definitions capture stable physical behaviour across turbines rather than patterns tied to a specific year.

The pipeline comparison shows that adding DQN as a first stage before the meta-model reduces readings flagged for human review from 4,189 (8.0% of all readings) in Pipeline A to 1,221 (2.3%) in Pipeline B, a reduction of about 70%. Pipeline B catches 98.9% of DQN's remaining errors with only 14 false alarms across 52,234 test readings.

The practical recommendation from this study is: for any deployment where labelled training data is available, a Random Forest or Gradient Boosting classifier is the most accurate and computationally efficient choice. DRL is the right choice when a labelling-free or continuously adapting solution is needed, with DQN being the recommended algorithm of the three tested.

Future work should test true online DRL adaptation on a live turbine, extend validation to multiple turbine types and wind farm sites, and measure the downstream impact of zone classification accuracy on fault detection and power curve modelling.

Declaration on the use of AI tools

During the preparation of this paper, AI tools (specifically ChatGPT and Claude by Anthropic) were used to assist with grammar correction, clarity improvement and general language refinement. The tools were applied conscientiously and in line with ethical guidelines. No AI was used to generate original ideas, analyses, or references. All research, analysis, interpretations, and conclusions are entirely the authors' own, and the authors take full responsibility for the content.

References

- [1] Manwell, J. F., McGowan, J. G. and Rogers, A. L. (2009). Wind energy explained: Theory, design and application (2nd ed.). John Wiley and Sons.
- [2] Indrekvam, G., Singh, M., Kampen, A.-L., Jha, M. S. (2026). Interpretable Operational Regime Classification for a Wind Turbine PHM Digital Twin Architecture. Proceedings of the 9th European Conference of the Prognostics and Health Management Society (PHME 2026).
- [3] Pandit, R. and Wang, J. (2024). A comprehensive review on enhancing wind turbine applications with advanced SCADA data analytics and practical insights. IET Renewable Power Generation, 18, 722-742.
- [4] Tavner, P. (2012). Offshore wind turbines: Reliability, availability and maintenance. The Institution of Engineering and Technology, IET Renewable Energy Series 13.
- [5] Cohen, W. W. (1995). Fast effective rule induction. In Proceedings of the 12th International Conference on Machine Learning (ICML), pp. 115-123. Morgan Kaufmann.
- [6] Quinlan, J. R. (1990). Learning logical definitions from relations. Machine Learning, 5(3), 239-266.
- [7] Pandit, R. K., Infield, D. and Kolios, A. (2019). Comparison of advanced non-parametric models for wind turbine power curves. IET Renewable Power Generation, 13(9), 1503-1510.
- [8] Xie, J., Dong, H., Zhao, X. (2023). Data-driven torque and pitch control of wind turbines via reinforcement learning. Renewable Energy, 215, 118893.
- [9] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533.
- [10] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.
- [11] Mnih, V., Badia, A. P., Mirza, M., Graves, A., Harley, T., Lillicrap, T., Silver, D. and Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In Proceedings of the 33rd International Conference on Machine Learning (ICML), pp. 1928-1937.
- [12] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. Journal of Machine Learning Research, 22(268), 1-8.
- [13] Towers, M., Terry, J. K., Kwiatkowski, A., et al. (2023). Gymnasium. Zenodo. <https://doi.org/10.5281/zenodo.8127025>
- [14] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems, 30, 3146-3154.
- [15] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.
- [16] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.

- [17] Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
- [18] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.